TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

# Module 9: KV-Cached Decoding

## *Reuse, Memory Growth, and Exact Autoregressive State*

**BRAIN team**

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

# Lecture map

1. From full-prefix recomputation to reuse
2. Cache memory and bandwidth
3. Correctness and benchmarking

**Guiding question**

What can autoregressive decoding reuse exactly, and how does that reuse turn compute into a memory-management problem?

# What should be clear by the end?

1. Explain the redundant work removed by a KV cache.
2. Compute cache shapes and memory for MHA, GQA, and MQA.
3. Reason about decode bandwidth and cache layout.
4. Test cached generation against a full-prefix reference.

PART 1

# From full-prefix recomputation to reuse

1. From full-prefix recomputation to reuse
2. Cache memory and bandwidth
3. Correctness and benchmarking

# Naive decoding recomputes the unchanged prefix

To generate token $t+1$, a naive decoder runs the model on $x_{0:t}$. The next step runs it again on $x_{0:t+1}$.

**Repeated work**

At every layer, keys and values for positions $0 \ldots t$ are deterministic functions of unchanged hidden states. Recomputing them cannot change the result.

**Cache principle**

Store past key and value projections once. At the next step, compute projections only for the new token and attend to cached history.

Source: Saad Ahmed, KV Cache Explained Intuitively

# Prefill and decode have different shapes

## Prefill

processes the prompt in parallel
many query tokens
large matrix multiplications
usually compute-efficient

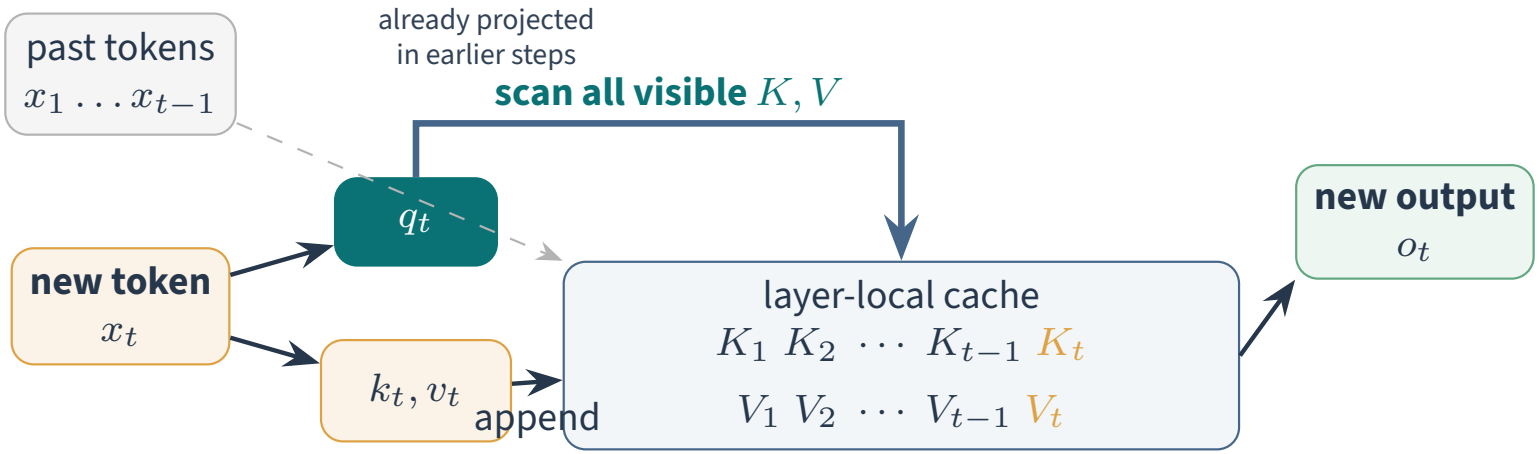
## Decode

processes one new token per request
reads growing KV history
small matrix operations
often bandwidth-bound

## Do not average them together

Time to first token and inter-token latency arise from different workloads and need separate measurements.

# Cached attention at one layer




$$o_t = \mathrm{softmax}\left(q_t K_{1:t}^\top / \sqrt{d_h}\right) V_{1:t}$$

The cache removes repeated $K, V$ projection; attention still scans the growing context.

# The cache changes asymptotic projection work

For $n$ generated tokens after a prompt:

**Without cache**

Reproject the growing prefix at every step. Prefix projection work accumulates roughly quadratically in generated length.

**With cache**

Project each token's key and value once. Attention still scans a context that grows linearly per decode step.

The cache removes redundant projection work. It does not make full-context attention constant-time.

PART 2

# Cache memory and bandwidth

---

1. From full-prefix recomputation to reuse
2. Cache memory and bandwidth
3. Correctness and benchmarking

# Cache shape determines capacity

For $L$ layers, batch $B$, $H_{kv}$ key-value heads, cached length $T$, head width $d_h$, and element size $s$:

$$M_{KV} = 2\,L\,B\,H_{kv}\,T\,d_h\,s$$

The factor 2 stores both keys and values.

**Per-token, per-request growth**

$$m_{\text{token}} = 2LH_{kv}d_h s$$

This number links model architecture directly to serving concurrency.

# Worked memory estimate

Suppose $L = 32$, $H_{kv} = 32$, $d_h = 128$, BF16, and $T = 4096$:

$$M_{KV} = 2 \cdot 32 \cdot 1 \cdot 32 \cdot 4096 \cdot 128 \cdot 2 \approx 2\,\text{GiB}$$

**Serving implication**

Sixteen concurrent requests at this cached length would need roughly 32 GiB for KV state alone, before weights, workspaces, and fragmentation.

# MHA, GQA, and MQA differ mainly in $H_{kv}$

| Variant | query heads | KV heads | cache effect |
|---|---|---|---|
| MHA | $H$ | $H$ | baseline |
| GQA | $H$ | $G$, where $1 < G < H$ | reduced by $H/G$ |
| MQA | $H$ | 1 | reduced by $H$ |

Query heads share key-value groups while retaining distinct queries. This reduces cache capacity and decode bandwidth at a possible modeling-quality trade-off.

# Decode often reads far more bytes than it computes

Each new token reads the cached $K, V$ for every active layer and request. Batch-1 decode has little reuse across queries.

## Bandwidth model

$$t_{\text{decode}} \gtrsim \frac{\text{weight bytes read} + \text{KV bytes read}}{BW_{\text{effective}}}$$

## Consequence

Reducing KV heads, using lower cache precision, batching requests, and improving memory layout can matter more than reducing arithmetic.

# Allocation strategy decides fragmentation and copy cost

### Concatenate

simple
repeated allocation and copying
poor for long generation

### Preallocate

fast append
reserves maximum length
internal waste

### Paged

non-contiguous blocks
flexible growth
mapping metadata

Paged layouts make variable-length request lifetimes manageable without requiring one contiguous maximum-size allocation per request.

# Checkpoint: capacity calculation

## Think, then compare

Recompute the worked example with eight KV heads. How many 4096-token requests fit in 32 GiB if 24 GiB is available for cache after weights and workspaces?

PART 3

# Correctness and benchmarking

---

1. From full-prefix recomputation to reuse
2. Cache memory and bandwidth
3. Correctness and benchmarking

# The cache must preserve logical positions

- The new token uses its absolute or rotary position $t$.
- The causal mask covers the valid cached prefix and current token.
- Padding and per-request lengths cannot expose uninitialized cache entries.
- Reordered or paged blocks must reconstruct the same logical sequence.

**Common failure**

Correct key-value values at the wrong positions produce plausible but divergent text.

# Full-prefix decoding is the reference oracle

For fixed weights, prompt, and token sequence:

1. run a full-prefix forward pass for each step
2. run cached prefill plus single-token decode
3. compare next-token logits at every step
4. repeat across batch sizes, prompt lengths, and cache boundaries

**Then test generation**

Greedy outputs should match exactly within numerical tolerance. Sampling tests require identical random state and probability inputs.

# Benchmark prefill and decode separately

| Phase | Primary metrics | Sweep |
|---|---|---|
| prefill | latency, prompt tokens/s, peak memory | prompt length, batch |
| decode | ms/token, output tokens/s, cache bytes | cached length, batch |
| end-to-end | TTFT, total latency | prompt/output distribution |

**Warm and synchronize**

Kernel compilation, allocation, and asynchronous timing can otherwise dominate or under-report short decode steps.

# Checkpoint: cache debugging

## Think, then compare

Cached logits match for short prompts but diverge immediately after a page boundary. Which state and indexing checks isolate layout bugs from positional-encoding bugs?

# The durable ideas

1. KV caching reuses past key and value projections exactly.
2. Cache memory grows with layers, KV heads, requests, cached tokens, head width, and precision.
3. Decode is commonly bandwidth-bound, so cache architecture and layout shape performance.
4. Full-prefix logit equivalence is the primary correctness test before benchmarking.